

MATH3302/MATH7332

Modern standards

NIST standards

The NIST cryptographic standards:

1. DES: The Data Encryption Standard (56-bit keys)
2. AES: The Advanced Encryption Standard (128, 192 or 256-bit keys)

History:

- 1977: DES standardised
- Late 1990s: DES looks increasingly weak
- 1999: Standard requires Triple DES for non-legacy systems
- 2002: AES standardised
- 2005: DES withdrawn

Good cipher design

Avalanche effect:

- Small parts of input → changes affect large parts of output

Completeness effect:

- Small parts of output → depend on large parts of input

Substitution-permutation networks

Block cipher: works on **blocks** of n bits

Uses successive substitutions and permutations

S-box: performs a substitution

- Bijection between all possible input and output **blocks**
- $(2^n)!$ bijections possible

P-box: performs a permutation (i.e., transposition)

- Bijection between input and output **bit positions**
- $n!$ bijections possible

DES: the Data Encryption Standard

- Developed at IBM
- 64-bit blocks, 56-bit key
- 16-round Feistel cipher (discussed in a later section)
- Main problem: 56-bit key is too small, now crackable well within a day
- Controversies with the NSA:
NSA replaced S-boxes, and reduced the key length
- In hindsight: the new S-boxes protected DES against differential cryptanalysis

AES: the Advanced Encryption Standard

Open competition for the new standard

The winner (which became AES) was **Rijndael**:
developed by Joan Daemen and Vincent Rijmen

128-bit blocks, several key sizes:

- 128-bit keys, 10 rounds
- 192-bit keys, 12 rounds
- 256-bit keys, 14 rounds

Today we will see the variant for 128-bit keys

AES: input and output

- Each block contains 128 bits
- Group these into 16 bytes (1 byte = 8 bits)
- Write each byte using two hexadecimal digits:
00111101 = 3d
11110101 = f5
- Pack all 16 bytes into a 4×4 state array,
one column at a time:

Hexadecimal	
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	a
1011	b
1100	c
1101	d
1110	e
1111	f

a7 8c b4 31 09 de 40 cb 10 7f 33 19 a5 7c ba b0 =

a7	09	10	a5
8c	de	7f	7c
b4	40	33	ba
31	cb	19	b0

The cipher

Subkeys (also called **round keys**):
 $K_0, K_1, \dots, K_r$

Each K_i is 128-bit, **deterministically** generated from the master key K

1. Add round key K_0
2. For each round $i = 1, 2, \dots, r-1$:
 - Substitute bytes
 - Shift rows
 - Mix columns
 - Add round key K_i

3. For the final round r :

- Substitute bytes
- Shift rows
- Add round key K_r

All addition of round keys is done using **bitwise XOR**

XOR is just **addition in $\mathbb{Z}_2$**

Shift rows

1st row: fixed
2nd row: cycle 1 step left
3rd row: cycle 2 steps left
4th row: cycle 3 steps left

a7	09	10	a5
8c	de	7f	7c
b4	40	33	ba
31	cb	19	b0



a7	09	10	a5
de	7f	7c	8c
33	ba	b4	40
b0	31	cb	19

Substitute bytes

- Replace each byte of the state array using a large S-box
- The S-box is based on inversion in the Galois field $GF(2^8)$ —we will discuss this shortly

The AES S-box

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_a	_b	_c	_d	_e	_f
0_	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1_	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2_	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3_	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4_	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5_	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6_	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7_	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8_	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9_	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a_	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b_	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c_	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d_	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e_	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f_	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Example: d1 $\mapsto$ 3e

Bytes as polynomials

Representing a single byte:

Hexadecimal	Bits	Polynomials over $\mathbb{Z}_2$ of degree < 8
3d	00111101	$x^5 + x^4 + x^3 + x^2 + 1$
f5	11110101	$x^7 + x^6 + x^5 + x^4 + x^2 + 1$
62	01100010	$x^6 + x^5 + x$

Bytes as elements of $GF(2^8)$

Use the Galois field $GF(2^8)$, defined as:

- Elements are polynomials of degree < 8 ,
with coefficients in $\mathbb{Z}_2$
- Addition:
Reduce coefficients modulo 2
- Multiplication:
Reduce coefficients modulo 2
Reduce polynomials modulo $x^8 + x^4 + x^3 + x + 1$
- The polynomial above is irreducible, and so we have a field!
We can divide by anything except for zero

Exercise

(a) Determine the polynomial forms for the bytes 5d and 31.

(b) Determine the product of these two polynomials in $GF(2^8)$ working modulo $x^8 + x^4 + x^3 + x + 1$.

Exercise Determine the inverse of the polynomial $p(x) = x^5 + x^4 + 1$
in $GF(2^8)$ working modulo $x^8 + x^4 + x^3 + x + 1$.

Building the S-box

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_a	_b	_c	_d	_e	_f
0_	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1_	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2_	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3_	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
⋮																

The substitution of a byte whose polynomial representation is $p(x)$ occurs in two steps:

1. Calculate $(p(x))^{-1}$ in $\text{GF}(2^8)$, with $0 \mapsto 0$, and let Y be the bitstring representation of $(p(x))^{-1}$.
2. The new byte is $Y + c_1(Y) + c_2(Y) + c_3(Y) + c_4(Y) + 01100011$, where addition is **bitwise**, and where $c_i(Y)$ means “cycle the bitstring Y , i bits to the left”.

Exercise Use the steps outlined above to compute the substitution for the byte 31, and then check your answer in the S-box.

AES recap

128-bit blocks

128-bit subkeys $K_0, K_1, \dots, K_r$

1. Add round key K_0

2. For each round $i = 1, \dots, r-1$:

- Substitute bytes
- Shift rows
- Mix columns
- Add round key K_i

3. For the final round r :

- Substitute bytes
- Shift rows
- Add round key K_r

Addition of round keys uses
bitwise XOR (addition in $\mathbb{Z}_2$)

Mix columns

For each column of the state array:

Using matrix multiplication over $\text{GF}(2^8)$:

$$\begin{array}{|c|} \hline b_0 \\ \hline b_1 \\ \hline b_2 \\ \hline b_3 \\ \hline \end{array} \mapsto \begin{array}{|c|c|c|c|} \hline x & x+1 & 1 & 1 \\ \hline 1 & x & x+1 & 1 \\ \hline 1 & 1 & x & x+1 \\ \hline x+1 & 1 & 1 & x \\ \hline \end{array} \times \begin{array}{|c|} \hline b_0 \\ \hline b_1 \\ \hline b_2 \\ \hline b_3 \\ \hline \end{array}$$

Example:

d4	e0	b8	1e
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5



04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

Exercise Determine the new entry in row 1, column 2 after Mix Columns is applied to the above example.

The key schedule

Generating round keys $K_0, K_1, \dots, K_r$ from the master key K uses:

- addition over $\mathbb{Z}_2$;
- substitution, using the same S-box as before;
- cyclic permutations.

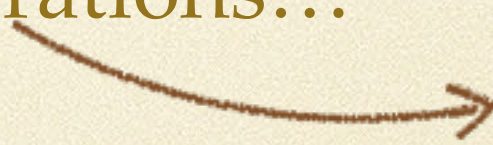
If you want the details, see the **full specification**:

- <http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf>

AES decryption

Same basic sequence of operations...

... but with minor tweaks:



same hardware,
same software

- Inverse S-box for “substitute bytes”
- Inverse matrix for “mix columns”
- Cycle in the other direction for “shift rows”
- Subkeys used in the reverse order, and need to have the inverse “mix columns” transformation applied

Modes of operation

ECB: Electronic codebook

- Encrypt blocks independently: $y_i = e_K(x_i)$

CFB: Cipher feedback

- Encryption uses previous ciphertext: $y_i = e_K(x_i + y_{i-1})$

OFB: Output feedback

- Use as a synchronous stream cipher: $y_i = x_i + \ell_i$
Keystream: $\ell_i = e_K(\ell_{i-1})$, ℓ_0 = initialisation vector

CBC: Cipher block chaining

- Use as an asynchronous stream cipher: $y_i = x_i + \ell_i$
Keystream: $\ell_i = e_K(y_{i-1})$, ℓ_0 = initialisation vector

Modes of operation

Electronic codebook:

- Bad: reuses plaintext-ciphertext pairs

Cipher block chaining & cipher feedback:

- Give a desirable butterfly / avalanche effect:
A small change in the plaintext will change
all of the remaining cipher text (not just one block)

Electronic codebook & output feedback:

- Good for error-prone environments
(can recover from errors in the ciphertext)

Public or private key?

Public key cryptography:

- Implementations typically require expensive mathematical operations
(algebra, number theory)

Private key cryptography:

- Implementations typically closer to native machine operations
(bit manipulation, substitution & permutation)

Typical usage:

- Use public key cryptography to agree on a secret key
- Use private key cryptography for the bulk of the data transfer